SPARK CORE FUNDAMENTALS

UnWrap concepts in SImplest way

SPARK JOIN STRATEGIES

JOIN STRATEGIES IN SPARK

Whenever we write Join statement in spark these joins will be implemented by spark internally in multiple ways/strategies.

Spark primarily have **5** joining strategies/ways

- Sort Merge Join Strategy
- Shuffle Hash Join Strategy
- Broadcast join strategy
- Broadcast Nested loop Join Strategy
- Cartesian Product Join Strategy

This choice of Join Strategy will be decided by spark automatically but understanding them helps to optimize the spark job performance in better way

The *Choice of selecting join strategy depends on*

- DataSize *(smaller or larger table)*
- Skewness in data
- Distribution ofdata *(howdataispartitioned across executors)*
- Optimization Goal *(minimal shuffling, memory usage)*

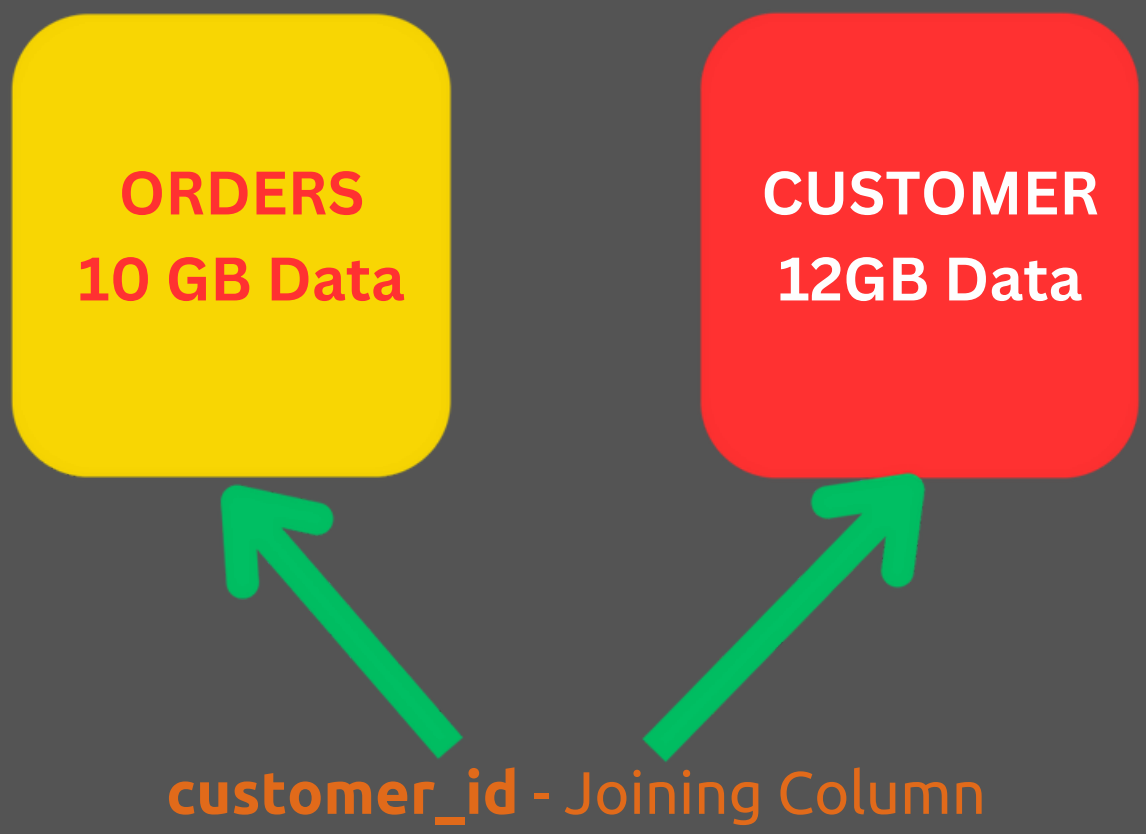
SHUFFLE HASH JOIN STRATEGY

Shuffle hash Join is a default join strategy in spark. this join strategy will be a better option when we have two medium sized or large tables and the join keys have high cardinality .

Shuffle Hash Join will be implemented in following steps

1. *Partitioning of data tables*
2. *Shuffling of data across tables*
3. *Hashing and In memory Join*
4. *Result Aggregation*

Let' s consider two data tables to understand each step



Let’ s see the data of these tables Orders and Customer

ORDERS

Order_id	Customer_Id	Amount
1	101	50
2	102	**100**
3	101	75
4	**103**	**60**
5	102	120

CUSTOMER

Customer_Id	Customer_Name
101	**John**
102	**Emma**
103	**Micheal**

Step - 1 Partitioning of Orders data table

Partitioning of **Order** Table

Order_id	Customer_id	Amount
1	101	50
3	101	75

Partition 1 on Executor 1

Order_id	Customer_id	Amount
2	102	50
5	102	75

Partition 2 on Executor 2

Order_id	Customer_id	Amount
4	103	60

Partition 3 on Executor 1

CREATION OF MULTIPLE PARTITIONS FROM ORDERS TABLE

Step - 1(b) Partitioniing of Customer Data table

Partitioning of **Customer** Table

Customer_Id	Customer_Name
101	John

Partition 1 on Executor 1

Customer_Id	Customer_Name
102	Emma

Partition 2 on Executor 2

Customer_Id	Customer_Name
103	Michael

Partition 3 on Executor 1

CREATION OF MULTIPLE PARTITIONS FROM CUSTOMER TABLE

After partitioning both tables, the partitions will be structured as follows :

EXECUTOR 1	EXECUTOR 2
(101, John)	(102,Emma)
(103, Michael)	
(1,101, 50)	(2,102, 100)
	(5,102, 120)
(4,103,60)	(3,101, 75)

PARTITIONING OF DATA

Step - 2 Data Shuffling

EXECUTOR 1	EXECUTOR 2
(101, John),(1,101,50),(3,101,75)	(102, Emma),(2,102,100),(5,102,120)
(103, Michael), (4,103,60)	

DATA SHUFFLING

In ***Shuffling***, Spark moves data across executors to ensure all row with the same customer_Id end up in the same partition on th same executor. Now, each executor has all relevant data for customer_Id.

Step - 3 Hash Table

The smaller table will have hash tables built for it.

Each executor constructs an in-memory hash table for the Customers table based on customer_Id.

Hash Table on Executor_1

EXECUTOR 1 Hash Table
(101, John)
(103, Michael)

Probe (Lookup) and Join:

(1,101, 50) → Matches John

(3,101, 75) → Matches John

(4,103, 60) → Matches Michael

FINAL OUTPUT ON EXECUTOR 1 -

orderId	customerId	amount	customerName
1	101	50	John
3	101	75	John
4	103	60	Michael

Hash Table on Executor_2

EXECUTOR 2 Hash Table

(102, Emma)

Probe (Lookup) and Join:

(2,102, 100) → Matches Emma

(5,102, 120) → Matches Emma

FINAL OUTPUT ON EXECUTOR 2-

orderId	customerId	amount	customerName
2	102	100	Emma
5	102	120	Emma

finally both the executors will be aggregated to get the complete output from Executor_1 and Executor_2

Step - 4 Result Aggregation

Results from both executors will be aggregated and final output will be

orderId	customerId	amount	customerName
1	101	50	John
3	101	75	John
2	102	100	Emma
5	102	120	Emma
4	103	60	Michael

FINAL OUTPUT AFTER JOINING ORDER AND CUSTOMER USING SHUFFLE HASH JOIN

Summary of Shuffle Hash Join

Hash Partitioning: Both tables are partitioned on customerId. Shuffling: Spark moves data so matching customerId are in the same executor. Hash Table Creation: Each executor builds a hash table for Customers using customerId. Join Execution: The Orders table is probed against the hash table for fast lookups. ***Final Output: The joined table is generated.***

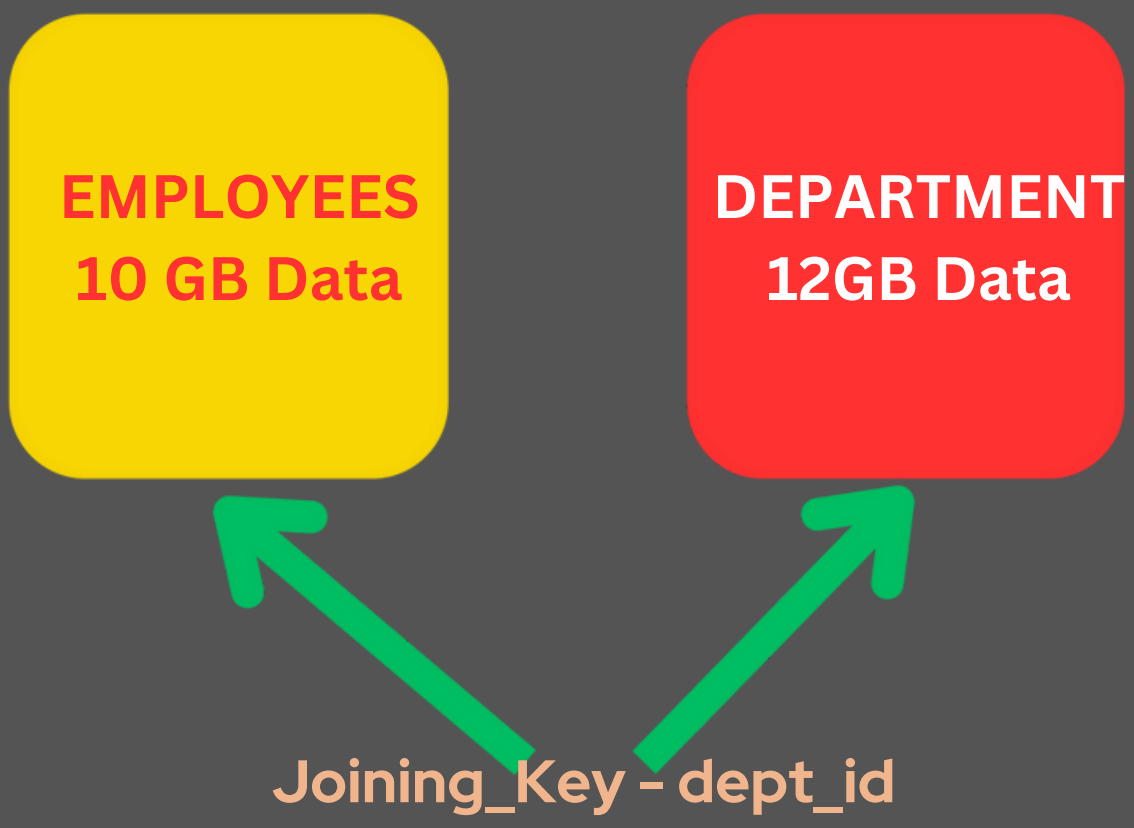
SORT MERGE JOIN STRATEGY

Shuffle sort merge join is a join strategy that combines elements of sorting and merging operations. It is effective when joining large datasets.

Shuffle sort merge join will be implemented in following steps :

1. *Partitioning of data tables*
2. *Shuffling of data across Tables*
3. *Sorting data based on join keys*
4. *Merge data step by step*

Let's consider two data tables to understand each step



Let' s see the data of these tables Employees and Department

EMPLOYEES

EMPLOYEES TABLE		
emp_id	name	dept_id
400	Michael	222
200	Abraham	111
800	John	555
1200	Peter	111
100	Kevin	333
300	Sarah	666
500	Nancy	222
1000	Robin	333
900	Jacob	666
700	Mark	444
600	James	555
1100	Smith	444

DEPARTMENT

DEPARTMENT TABLE	
dept_id	dept_name
444	dept4
222	dept2
666	dept6
111	dept1
333	dept3
555	dept5

Step - 1 Partitioning of data tables

Let's assume we have three partitions within one worker node then partitions will be

PARTITION 1		
Employees Table Partition		
emp_id	name	dept_id
400	Michael	222
200	Abraham	111
800	John	555
1200	Peter	111
Department Table Partition		
dept_id	dept_name	
444	dept4	
222	dept2	

Partition 1 on Executor 1

PARTITION 2		
Employees Table Partition		
emp_id	name	dept_id
100	Kevin	333
300	Sarah	666
500	Nancy	222
1000	Robin	333
dept_id	dept_name	
666	dept6	
111	dept1	

Partition 2 on Executor 2

PARTITION 3		
Employees Table Partition		
emp_id	name	dept_id
900	Jacob	666
700	Mark	444
600	James	555
1100	Smith	444
Department Table Partition		
dept_id	dept_name	
333	dept3	
555	dept5	

Partition 3 on Executor 3

Since the partitions are distributed across multiple executors, *each partition currently contains records with* ***different dept_id*** *values*, leading to mismatches within partitions.

To resolve this issue, we will perform data shuffling across partitions*, ensuring that all records with the same dept_id are grouped within the same partition.*

Step - 2 Shuffling of data tables partitions

After Shuffling of three partitions within one worker node then partitions will be

PARTITION 1 AFTER SHUFFLING		
Employees Table Partition		
emp_id	name	dept_id
400	Michael	222
200	Abraham	111
1200	Peter	111
500	Nancy	222
Department Table Partition		
dept_id	dept_name	
222	dept2	
111	dept1	

Partition 1 AFTER SHUFFLING

PARTITION 2 AFTER SHUFFLING		
Employees Table Partition		
emp_id	name	dept_id
100	Kevin	333
1000	Robin	333
700	Mark	444
1100	Smith	444
Department Table Partition		
dept_id	dept_name	
444	dept4	
333	dept3	

Partition 2 AFTER SHUFFLING

PARTITION 3 AFTER SHUFFLING		
Employees Table Partition		
emp_id	name	dept_id
900	Jacob	666
600	James	555
800	John	555
300	Sarah	666
Department Table Partition		
dept_id	dept_name	
555	dept5	
666	dept6	

Partition 3
AFTER SHUFFLING

After shuffling, each executor now has matching dept_id values within each partition. However, the data is still not properly sorted based on the joining key (dept_id). **This issue exists across all partitions.**

In the next step, we will perform **sorting** within each partition based on dept_id. Once sorting is complete, we will merge the sorted partitions to produce the final output.

PARTITION 1 AFTER SHUFFLING AND SORTING			PARTITION 2 AFTER SHUFFLING AND SORTING			PARTITION 3 AFTER SHUFFLING AND SORTING		
Employees Table Partition			Employees Table Partition			Employees Table Partition		
emp_id	name	dept_id	emp_id	name	dept_id	emp_id	name	dept_id
200	Abraham	111	1000	Robin	333	600	James	555
1200	Peter	111	100	Kevin	333	800	John	555
400	Michael	222	700	Mark	444	900	Jacob	666
500	Nancy	222	1100	Smith	444	300	Sarah	666
Department Table Partition			Department Table Partition			Department Table Partition		
dept_id	dept_name		dept_id	dept_name		dept_id	dept_name	
111	dept1		333	dept3		555	dept5	
222	dept2		444	dept4		666	dept6	

ALL THREE PARTITIONS DATA AFTER SORTING ON DEPT_ID

Step - 3 Merging of data tables partitions

PARTITION 1 AFTER SHUFFLING, SORTING AND MERGING			
EMPLOYEES AND DEPARTMENT MERGED Partition 1			
emp_id	name	dept_id	dept_name
200	Abraham	**111**	dept1
1200	Peter	111	dept1
500	Nancy	222	dept2
400	Michael	**222**	dept2

Partition 1 AFTER SHUFFLING, SORTING AND MERGING

PARTITION 2 AFTER SHUFFLING, SORTING AND MERGING			
EMPLOYEES AND DEPARTMENT MERGED Partition 1			
emp_id	name	dept_id	dept_name
100	Kevin	**333**	dept3
1000	Robin	333	dept3
700	Mark	444	dept4
1100	Smith	**444**	dept4

Partition 2 AFTER SHUFFLING, SORTING AND MERGING

PARTITION 3 AFTER SHUFFLING, SORTING AND MERGING			
EMPLOYEES AND DEPARTMENT MERGED Partition 1			
emp_id	name	dept_id	dept_name
600	James	**555**	dept5
800	John	555	dept5
300	Sarah	666	dept6
900	Jacob	**666**	dept6

Partition 3 AFTER SHUFFLING, SORTING AND MERGING

Since the shuffle, sort, and merge steps have been completed, the final step involves combining the resulting data partitions within worker node to generate the final output.

Once we combine the data from all three partitions, it will be structured as follows:

FINAL RESULT SET DATA

FINAL RESULT PARTITION_1 + PARRTITION_2 + PARTITION_3			
FINAL RESULT SET			
emp_id	**name**	**dept_id**	**dept_name**
400	Michael	**222**	dept2
200	Abraham	**111**	dept1
800	John	**555**	dept5
1200	Peter	**111**	dept1
100	Kevin	**333**	dept3
300	Sarah	**666**	dept6
500	Nancy	**222**	dept2
1000	Robin	**333**	dept3
900	Jacob	**666**	dept6
700	Mark	**444**	dept4
600	James	**555**	dept5
1100	Smith	**444**	dept4

This is how a **Sort-Merge Join** works by shuffling, sorting, and merging data efficiently across multiple partitions.

BROADCAST JOIN STRATEGY

Broadcast join is a join strategy that is supposed to be used when we have one small table and with large table

In Broadcast Join, the smaller data table will be broadcasted to all the worker nodes of larger data table.

Broadcasting the smaller data table to all partitions/workers of big table will reduce the data shuffling and each worker can perform join locally

Let's see how a broadcast join works

Suppose you have two data files to process -

file1 - 6GB (Big file)
file2 - 1GB (Small file)

both these files has been partitioned on as

file1 - 3 partitions 2GB each on 3 executors/workers
file2 - small file no partitions

.

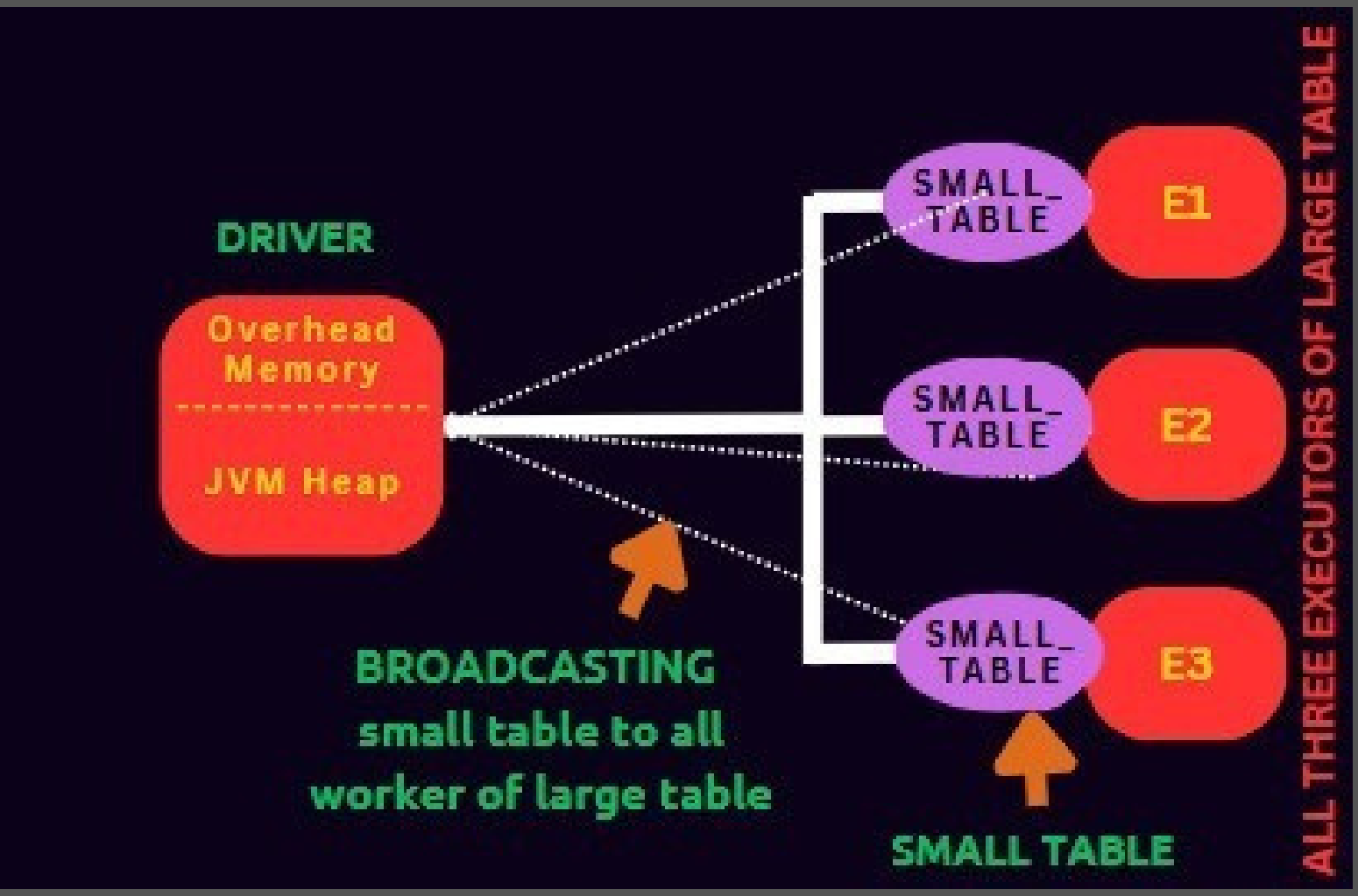


DIAGRAM – BROADCAST JOIN – BROADCASTING SMALL TABLE COPY TO ALL WORKERS OF LARGE TABLE

As shown in above diagram file1 (6GB) is big and other file2(1GB) is small. In such scenarios we will surely think for broadcast join and broadcasting small data table to big file partitions to reduce the shuffling of data.

(Above is the concept of broadcast join)

The small table is processed in parallel on each worker node without any shuffling, and the results are later merged to produce the final output.

This is how a Broadcast Join works, eliminating the need for data shuffling across multiple executors and workers.

However, it is only effective when one table is small enough to be efficiently broadcasted across the worker

Key Advantages of Broadcast Join -

- ✅ Avoids expensive data shuffling, reducing network overhead.
- ✅ Faster execution, as small table data is available locally on each node.
- ✅ Ideal for lookups, enriching large datasets with reference data.